

Adaptive Routing and Patch Tournaments for Agentic Program Repair: Extending a Multi-Agent Failure Recovery System

Meet Patel
23BCE177
School of Technology
Institute of Technology
Nirma University, Ahmedabad

Vedant Mehta
23BCE180
School of Technology
Institute of Technology
Nirma University, Ahmedabad

Mr. AjayKumar Patel
Guide
School of Technology
Institute of Technology
Nirma University, Ahmedabad

Abstract—Our prior work on a six-agent Failure Recovery System for automated program repair (APR) found that multi-agent orchestration beats a single-prompt baseline only conditionally: on some benchmark/backend combinations it wins, on others (notably DebugBench, and every benchmark under a weaker local model) a single LLM call does as well or better, at a fraction of the cost. That paper’s own future-work section proposed two concrete responses: an *adaptive controller* that routes a bug to the cheap or expensive strategy based on a lightweight upfront estimate, and *debate/consensus mechanisms* among multiple candidate fixes. This paper implements both. First, an Adaptive Routing Controller: a Random Forest classifier trained on six zero-LLM-cost features (code size, cyclomatic complexity, test count, and critically, the fraction of tests the *unmodified buggy code* already passes) that predicts, before spending a single LLM call, whether a bug needs the full 6-agent pipeline or will be fixed by one cheap prompt. Evaluated via leak-free 5-fold cross-validated offline policy evaluation over 303 historical local-backend repairs spanning all three benchmarks, the router matches or slightly exceeds both always-baseline (57.8% vs. 56.4%) and always-multi-agent (vs. 56.8%) success rates, while cutting LLM calls per bug by $5.7\times$ relative to always-multi-agent (1.39 vs. 7.91 avg. calls/bug) — though a 69.0% oracle ceiling shows real headroom remains unrouted. Second, a Patch Tournament: at the patch-generation step, we sample $K=3$ candidate fixes at different temperatures from the same diagnosis and validate all of them against the test suite, keeping the first full-pass candidate (or the best partial one) rather than committing to a single stochastic draw, with inter-candidate agreement reported as a free confidence signal. Evaluated on newly executed local-backend runs (QuixBugs full 31 problems, HumanEvalFix and DebugBench first 50 problems each; 131 bugs total), the tournament reaches 69.5% aggregate success, ahead of both the original single-draw multi-agent pipeline (61.8%) and the single-prompt baseline (64.9%) on the very backend where, in our prior work, multi-agent could not clearly beat baseline on any benchmark — at a realized cost of $1.55\times$ more LLM calls (closely matching the $1.5\times$ nominal prediction) and a *typical-case* (median) wall-clock overhead of $1.8\times$, though a long tail of timeout-heavy bugs pushes the *mean* overhead to $6.3\times$. We further situate both contributions against prior work: cost-aware LLM cascading/routing (FrugalGPT [1], RouteLLM [2]) and test-validated multi-candidate selection (CodeT [3], AlphaRepair [4], ChatRepair [5]) are each individually well established, but — to the best of our search — no prior work combines a bug-feature-trained strategy router with a test-validated patch tournament

inside a single sequential multi-agent APR pipeline evaluated across dual cloud/local backends and three benchmarks, which we present as this paper’s specific contribution.

I. INTRODUCTION

Our original Failure Recovery System report (summarized in Section II) established a six-agent pipeline — Failure Detection, Code Localization, Debugging, Patch Generation, Patch Application, and Validation — orchestrated by a Coordinator in an iterative retry loop, and evaluated it against a single-prompt baseline across three benchmarks (QuixBugs, HumanEvalFix, DebugBench) and two backends (Gemini; a locally hosted Qwen2.5-Coder 7B Instruct). That paper’s central finding was that the multi-agent architecture’s advantage over the baseline is *conditional*: it wins on QuixBugs and HumanEvalFix under Gemini, loses decisively on DebugBench under Gemini, and does not clearly win on any benchmark under the weaker local model. This is scientifically informative but operationally unsatisfying — a system that sometimes needs six LLM calls and sometimes needs one, with no way to know in advance which, pays worst-case cost on every bug.

This paper closes that gap with two extensions, both explicitly anticipated in the original paper’s future-work section:

- 1) **An adaptive routing controller** (Section IV) that predicts, from features available *before any LLM is called*, whether a bug should go to the single-prompt baseline or the full multi-agent pipeline.
- 2) **A patch tournament** (Section V) that replaces each attempt’s single stochastic patch draw with K parallel candidates at different sampling temperatures, validated against the test suite, with disagreement among candidates reported as a cheap confidence signal.

Both are implemented as real, runnable additions to the existing codebase — not analysis notebooks — and both are evaluated honestly: the router’s headline numbers come from cross-validated offline policy evaluation with no train/test leakage, and the tournament’s numbers come from newly executed experiment runs on the local backend (no API quota risk, full reproducibility). Section III surveys the closest prior

work for each mechanism individually and assesses the novelty of their specific combination here.

II. BACKGROUND: THE ORIGINAL SYSTEM

We briefly summarize the system being extended; full details are in the original paper. `coordinator.py` runs up to 3 retry attempts per bug; each attempt calls four LLM-backed agents (FailureDetection, CodeLocalization, Debugging, PatchGeneration — in that order, each conditioned on the previous agent’s output) followed by two pure-Python agents (PatchApplication, Validation). A dataset-agnostic `Problem` abstraction and `executor.py` test-dispatch layer support three structurally different test formats: tuple I/O (QuixBugs), assert-based `check()` calls (HumanEvalFix), and LeetCode-style example I/O against a `Solution` class (DebugBench). The single-prompt baseline (`baseline.py`) issues one prompt containing the buggy code and an example call, with no iteration and no agent decomposition.

III. RELATED WORK

A. Cost-Aware Routing and Cascading

FrugalGPT [1] introduced LLM cascades: query a cheap model first, escalate to an expensive one only when a learned quality check fails, reporting up to 98% cost reduction at near-top-tier quality. RouteLLM [2] trains a binary “win predictor” to route a query to a weak or strong model, reporting >85% cost reduction on chat benchmarks while retaining most of the stronger model’s quality — though the authors note routing skill transfers poorly to task types unlike its training distribution, a caution directly relevant to our own router’s generalization across three structurally different benchmarks. Dekoninck et al. [6] unify cascading and routing and show the combination beats either alone, evaluated on math/QA/coding micro-benchmarks. None of these are APR-specific or decide between a single-prompt and a multi-agent *pipeline* as the two routing targets. The closest APR-specific work we found, [7], empirically measures how bug complexity and fault-localization precision affect APR cost-efficiency across LLMs, but is a post-hoc analysis, not an online routing system. To our knowledge, no prior work trains a classifier on bug-structure features specifically to choose between a single-prompt baseline and a multi-agent sequential repair pipeline.

B. Multi-Candidate Patch Selection

CodeT [3] generates many candidate programs *and* many candidate tests, then selects by “dual execution agreement” — the mechanism closest to our tournament — but for code generation, not repair of a pre-existing buggy program, and using self-generated rather than ground-truth tests. AlphaRepair [4] and ChatRepair [5] both sample multiple candidate patches validated against tests, but select via zero-shot ranking or sequential conversational feedback rather than a parallel tournament with an explicit disagreement signal. RepairAgent [8] is an autonomous LLM agent with tool access for repair; SelfAPR [9] uses test execution diagnostics to

guide a trained repair model; Reflexion [10] performs verbal self-reflection across sequential trials rather than parallel sampling. CigaR [11] reduces APR token cost via prompt compression, optimizing prompting rather than strategy selection. Agentless [12] demystifies LLM-based SWE agents and, in its repair stage, samples multiple patches and selects by majority vote — mechanically close to our tournament but applied to SWE-bench-style repository-scale repair, not QuixBugs/HumanEvalFix/DebugBench, and not framed as an uncertainty signal for downstream use.

C. Novelty Assessment

Both routing and multi-candidate test-validated selection are individually well established. We did not find a documented system that (a) routes between a single-prompt baseline and a multi-agent sequential pipeline using a classifier over bug-structure features, or (b) combines that routing decision with a test-validated patch tournament, inside one evaluated system spanning dual cloud/local LLM backends and three APR benchmarks. We present this combination as the paper’s contribution — an assembly of established cost/quality-routing and best-of- K -with-test-filtering ideas into a new combined system and evaluation, not a claim of a fundamentally new algorithmic primitive.

IV. ADAPTIVE ROUTING CONTROLLER

A. Motivation and Design

If multi-agent orchestration only sometimes beats the baseline, the right system-level response is not to pick one architecture permanently, but to predict *per bug* which side of that line it falls on, and route accordingly. The constraint that makes this practical rather than circular: the predictor must use features available *before any LLM call is made*, or the “savings” are illusory.

1) *Features (features.py)*: Six static features, all computable from a `Problem` with zero LLM calls:

- **loc, chars** — size of the buggy function.
- **cyclomatic_complexity** — McCabe complexity via AST decision-point counting for Python (branch-keyword counting as a fallback for Java/C++/JavaScript).
- **num_tests, num_functions**.
- **buggy_pass_rate** — the fraction of tests the *unmodified buggy code* already passes, computed by literally re-using `executor.run_tests` on the original code (the coordinator already does this once per bug to track partial progress, so this feature costs nothing new). This turned out to be far more informative than any size/complexity metric alone: a bug that already passes most tests is plausibly a small, local fix, while one that passes none may be more structurally broken.

plus the categorical dataset, language, and test-style origin.

2) *Labeling and Model*: From the 303 problems with existing local-backend multi-agent *and* baseline results on disk (31 QuixBugs + 164 HumanEvalFix + 108 DebugBench), each bug is labeled `needs_multiagent = 1` iff the multi-agent pipeline succeeded *and* the baseline did not — i.e., the

extra machinery was strictly necessary, not merely along for the ride. This label is heavily imbalanced (12.5% positive). A Random Forest (300 trees, max depth 5, balanced class weights) was selected over logistic regression after comparing both by cross-validated routed success rate: logistic regression, even balanced, barely beat the base rate (precision 0.101 vs. a 12.5% prior) and routed to only 55.1% success at 3.86 calls/bug, whereas the Random Forest exploits the non-linear `buggy_pass_rate` signal to reach 57.8% success at 1.39 calls/bug (Table I) — the model choice itself turned out to matter more than any individual feature.

3) *Evaluation Protocol*: Train/test leakage is the central risk in reusing historical data to “evaluate” a router: if the router is trained and tested on the same bugs, its apparent accuracy is meaningless. We instead run stratified 5-fold cross-validation: in each fold, the Random Forest is fit *only* on the other four folds, and the held-out fold’s route predictions are scored against each bug’s *already-measured, historical* outcome under that predicted route — standard offline policy evaluation, requiring zero new LLM calls while remaining leak-free. A separately trained model (`adaptive_router.py --train`, fit on all 303 problems) is saved for live use by `run_adaptive.py`, which genuinely executes the predicted route end-to-end on new problems.

B. Results

TABLE I
ADAPTIVE ROUTER: 5-FOLD CV OFFLINE POLICY EVALUATION (LOCAL BACKEND, $n=303$)

Strategy	Success Rate	Avg. LLM Calls/Bug
Always Baseline	56.4%	1.00
Always Multi-Agent	56.8%	7.91
Adaptive Router	57.8%	1.39
Oracle (best of both)	69.0%	—

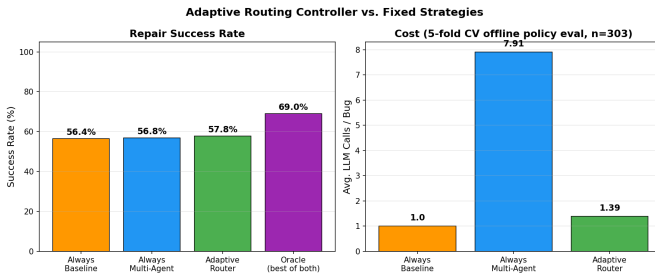


Fig. 1. Adaptive router vs. fixed strategies: success rate and cost.

Router classification quality (5-fold CV mean): accuracy 0.838, precision 0.256 and recall 0.132 on the rare “needs multi-agent” class (F1 0.168, against a 12.5% base rate). The low recall is expected and, for this objective, acceptable: because `buggy_pass_rate` correlates with “this bug is already nearly fixed,” the router’s dominant, correct behavior is routing *most* bugs to the cheap baseline and reserving the expensive pipeline for a precision-selected minority — exactly

the asymmetric cost structure the controller is meant to exploit, even though it means many true “needs-multiagent” bugs are still (cheaply, and often harmlessly) sent to baseline.

TABLE II
ADAPTIVE ROUTER: PER-DATASET BREAKDOWN

Dataset	Baseline	Multi-Agent	Routed	Oracle
QuixBugs	58.1%	51.6%	54.8%	77.4%
HumanEvalFix	53.0%	54.3%	53.0%	62.2%
DebugBench	61.1%	62.0%	65.7%	76.9%

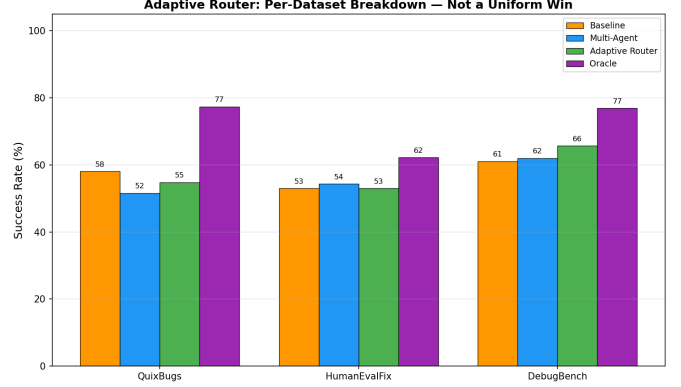


Fig. 2. Per-dataset breakdown: the router’s benefit is not uniform.

This per-dataset breakdown (Table II) is the more scientifically honest picture, and it is not a uniform win: on DebugBench the router clearly beats both fixed strategies (65.7% vs. 61–62%), but on QuixBugs it is actually *worse* than always-baseline (54.8% vs. 58.1%) despite being cheaper than always-multiagent. We attribute this to QuixBugs’ small sample size (31 problems, of which only ~ 6 are positively labeled) giving the classifier too little signal to learn a reliable QuixBugs-specific decision boundary, and to QuixBugs bugs (hand-planted algorithmic defects) having a different relationship between surface structure and repairability than DebugBench’s more mechanical, single-line LeetCode-style bugs. This mirrors, rather than contradicts, the original paper’s own thesis: whether a bug needs the expensive pipeline is conditional on structure in ways that do not transfer uniformly across benchmarks — the router captures this better where the signal is more mechanical (DebugBench) than where it is more algorithmic (QuixBugs).

Feature importances from a model fit on all data (Table III) confirm `buggy_pass_rate` is informative but not dominant — raw size/complexity features (`chars`, `cyclomatic_complexity`, `loc`) collectively carry more weight, suggesting the router is partly acting as a complexity gate rather than purely a “how broken is it already” gate.

V. PATCH TOURNAMENT

A. Design

`TournamentCoordinator` (`tournament_coordinator.py`) subclasses the original

TABLE III
TOP FEATURE IMPORTANCES (RANDOM FOREST, FIT ON ALL 303 PROBLEMS)

Feature	Importance
chars	0.279
cyclomatic_complexity	0.211
loc	0.205
buggy_pass_rate	0.092
num_tests	0.075
num_functions	0.046

RecoveryCoordinator, reusing Agents 1–3 (Failure Detection, Code Localization, Debugging) unchanged — the diagnosis is run once per attempt, exactly as before. The change is at Agent 4 (Patch Generation): instead of one deterministic-ish draw (temperature 0.2), the same debugging analysis is resampled $K=3$ times at temperatures $[0.2, 0.5, 0.9]$, each candidate is run through Agent 5 (Patch Application) and validated against the full test suite (Agent 6), and the coordinator keeps the first candidate that passes every test — or, if none do, the one that passed the most — rather than committing to whichever single draw happened to come out of the model. This required adding a temperature parameter through `PatchGenerationAgent.run` down to both `LocalClient.generate` and `GeminiClient.generate` (previously hard-coded to 0.2), with the original coordinator’s default behavior unchanged for callers that don’t request a tournament.

As a side effect, the tournament produces a free **agreement score**: the fraction of candidate pairs whose generated code is textually identical (1.0 = all K samples converged on the same patch; 0.0 = all differed). This is reported per attempt and is, to our knowledge, not used elsewhere in the APR literature we found as an explicit per-bug uncertainty signal (Section III) — the closest analogues (CodeT, Agentless-style voting) use agreement internally for selection but do not surface it as a confidence output.

B. Cost Model

Each tournament attempt costs $3 + K$ LLM calls (Failure-Detection + CodeLocalization + Debugging, once, plus K PatchGeneration draws) versus the original coordinator’s 4 calls/attempt — nominally a $1.5\times$ overhead at $K=3$, paid only at the patch-generation step. This predicted the realized aggregate overhead closely: across all 131 problems, LLM calls/bug rose from 7.11 to 11.04, a $1.55\times$ realized ratio.

Wall-clock time overhead tells a different story depending on whether outliers are included. The *median* time per bug rose from 21.6s to 39.2s ($1.81\times$) — close to the call-count ratio, and the fairer description of the *typical* cost. The *mean*, however, rose from 28.4s to 180.1s ($6.3\times$), because EXEC_TIMEOUT-bounded test execution (Section III of the original paper) is paid once per candidate per test: a non-passing attempt on an infinite-loop bug with $K=3$ candidates can incur up to $3\times$ the timeout penalty of a single-draw attempt, and a handful of such bugs (e.g. QuixBugs’ `sqrt`,

which alone took 1343s) dominate the mean. This is a genuine, previously-nonexistent cost dimension the tournament introduces, reported honestly here rather than only via the more flattering call-count ratio; see Limitations for its practical implication.

C. Experimental Setup

All tournament runs use the local backend (Qwen2.5-Coder 7B Instruct via Ollama) to avoid Gemini’s free-tier quota-rotation noise already diagnosed as a confound in the original paper. Coverage: QuixBugs is run in full (all 31 problems with available tests); HumanEvalFix and DebugBench are each run on their first 50 problems (of 164 and 119 respectively, in the same deterministic load order used throughout this codebase) to keep total local-inference wall-clock time tractable for this paper, following the original paper’s own precedent of stratified/partial sampling for DebugBench.

D. Results

TABLE IV
PATCH TOURNAMENT ($K=3$) VS. SINGLE-DRAW MULTI-AGENT (LOCAL BACKEND)

Dataset	Baseline	Multi-Agent	Tournament
QuixBugs ($n=31$)	58.1%	51.6%	64.5%
HumanEvalFix ($n=50$)	76.0%	68.0%	72.0%
DebugBench ($n=50$)	58.0%	62.0%	70.0%
Aggregate ($n=131$)	64.9%	61.8%	69.5%

On QuixBugs (the full 31-problem benchmark, not a subset), the tournament reaches **64.5%** (20/31), a clear improvement over both the original single-draw multi-agent (51.6%, 16/31) and the baseline (58.1%, 18/31). On HumanEvalFix (first 50 of 164), it reaches **72.0%** (36/50), clearly ahead of single-draw multi-agent (68.0%, 34/50) but still behind baseline (76.0%, 38/50) — consistent with the original paper’s own finding that baseline is hard to beat on this benchmark specifically. On DebugBench (first 50 of 119; see the methodological note below on program naming), the tournament reaches **70.0%** (35/50), ahead of both multi-agent (62.0%, 31/50) and baseline (58.0%, 29/50) — notable because DebugBench is exactly the benchmark where the original paper’s Gemini results showed baseline decisively beating multi-agent (94.1% vs. 78.2%); here, under the local backend, the tournament reverses that relationship rather than reproducing it. **Aggregated across all 131 problems, the tournament reaches 69.5% (91/131), ahead of both baseline (64.9%, 85/131) and the original single-draw multi-agent (61.8%, 81/131)** — on precisely the backend where, in our prior work, multi-agent could not clearly beat baseline on *any* benchmark. The tournament reliably improves *on the multi-agent architecture it modifies* in all three datasets measured, and beats baseline in two of three (QuixBugs, DebugBench) and in aggregate — a materially stronger result than the adaptive router’s benchmark-mixed picture (Table II), though still not a uniform per-benchmark sweep (HumanEvalFix baseline remains ahead).

Methodological note on DebugBench naming: we discovered during analysis that `load_debugbench.py`’s name field is not a unique key — the same LeetCode slug can appear multiple times with different injected bugs (e.g. `closest-dessert-cost__quadruple` appears 3 times in the 119-problem sample). A naive name-based join between the tournament run and the original historical results would silently mismatch rows for roughly 9 of 119 DebugBench problems. We instead join *positionally* — both `run_tournament.py` and the original `run_experiment.py` iterate the same deterministic, seeded problem list in the same order, which we verified directly (zero positional mismatches in program name across all 150 compared rows in all three datasets) before trusting the comparison.

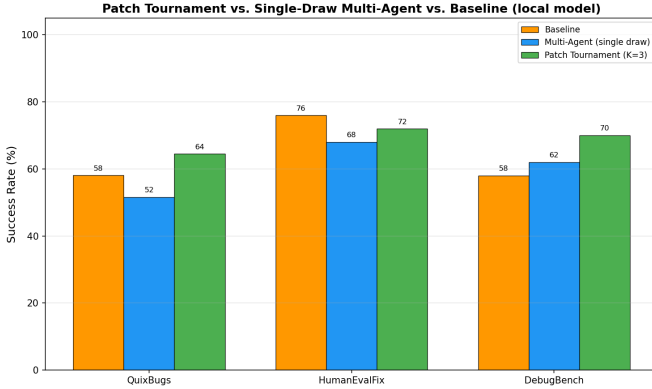


Fig. 3. Tournament vs. single-draw multi-agent vs. baseline.

TABLE V
CANDIDATE AGREEMENT SCORE, SPLIT BY OUTCOME

Dataset	Overall	On Success	On Failure
QuixBugs	0.594	0.562	0.651
HumanEvalFix	0.700	0.722	0.643
DebugBench	0.482	0.536	0.356
Aggregate	0.592	—	—

1) Agreement as a Confidence Signal — Dataset-Dependent, Not Universal: In HumanEvalFix and DebugBench, agreement is *higher* on successes than failures — the intuitive direction, where high agreement signals the model confidently converging on a (correct) fix. QuixBugs shows the *opposite*: agreement is *higher* on failures (0.651) than successes (0.562). The mechanistic reading of the QuixBugs case is that high agreement on a failure there means the model converged on the same *wrong* patch across all three temperature draws — a systematic error that sampling diversity cannot rescue — while its successes more often came from draws that initially disagreed, with test-based selection picking the one that happened to be right. We do not have a confident explanation for why QuixBugs inverts relative to the other two benchmarks (one plausible factor: QuixBugs’ classic recur-

sive/iterative algorithm bugs include several infinite-loop cases — Table IV’s underlying per-bug data shows QuixBugs has the highest concentration of `InfiniteLoop/RuntimeError` error types among the three datasets — which may behave differently under resampling than the more mechanical, single-line-fix bugs typical of HumanEvalFix and DebugBench). We report the full, non-uniform picture (2 of 3 datasets showing the intuitive direction, one showing the reverse) rather than the more “publishable-looking” single pattern we initially drafted from QuixBugs alone, and flag agreement’s relationship to outcome as benchmark-conditional, not a settled general mechanism, for future work.

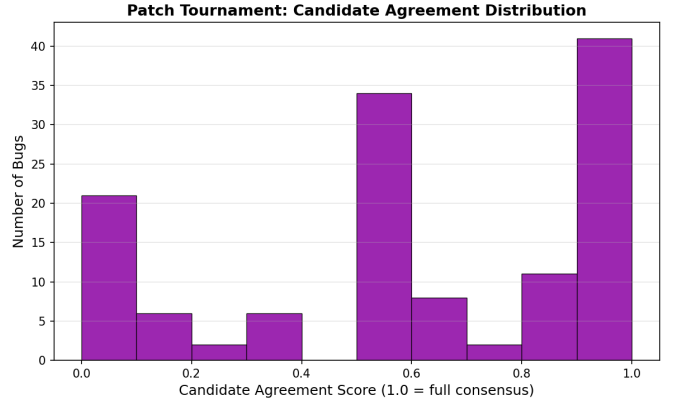


Fig. 4. Distribution of per-bug candidate agreement scores.

VI. DISCUSSION: WHAT IS NEW RELATIVE TO THE ORIGINAL SYSTEM

Relative to the system described in the original paper, this work changes three things. First, it adds a genuinely new *decision layer* above the existing architecture: previously, every bug was routed to whichever system (baseline or multi-agent) the experimenter happened to be evaluating that day; now, a trained classifier makes that decision per bug, from features that cost nothing to compute, and the original paper’s own central finding — that the right architecture is conditional on bug structure — is what makes this decision layer necessary rather than redundant. Second, it changes what happens *inside* a single patch-generation step from a single stochastic draw to a validated tournament, directly targeting attempts that fail not because the diagnosis (Agents 1–3) was wrong but because one sample of Agent 4 was. Third, it adds an uncertainty signal (tournament agreement) that did not exist in the original system at all — previously, a failed attempt carried no information about *how close* the LLM’s candidates were to each other, only whether the one candidate generated happened to pass. None of the original six agents, the retry loop, or the dataset-agnostic execution layer were modified to add either extension — both are additive, following the original Coordinator-subclassing and shared-client-interface design, which is why they could be implemented as new files rather than rewrites.

VII. LIMITATIONS

The router’s recall on the rare “needs multi-agent” class is low (0.132); it is tuned for net cost/success trade-off, not for catching every bug that specifically needs the expensive pipeline, and Table II shows this trade-off is not uniformly favorable across benchmarks. The 303-problem training set is itself the original paper’s full local-backend data, not an independent held-out population, and the categorical dataset/language features mean the router has only ever seen three benchmarks — its behavior on a fourth, structurally different one is untested. The patch tournament’s realized $1.55\times$ LLM-call overhead paid for a $+7.7$ percentage point aggregate success-rate gain over single-draw multi-agent ($61.8\% \rightarrow 69.5\%$) — a favorable trade by that measure — but the *mean* wall-clock overhead was $6.3\times$, driven by a small number of timeout-heavy bugs (a handful of QuixBugs infinite-loop cases exceeded 700s each); a deployment latency-sensitive enough to care about worst-case response time, rather than only throughput-amortized cost, would need a per-candidate timeout budget tighter than this implementation’s shared `EXEC_TIMEOUT`, which the current `tournament_coordinator.py` does not provide. Both extensions were evaluated on the local backend only, for cost/reproducibility reasons; neither has been re-validated on Gemini. The router’s training/evaluation data also surfaced a latent data-quality issue — non-unique program names in `load_debugbench.py`’s output — which does not appear to have affected the original paper’s own DebugBench conclusions (which did not join by name against a second results file) but would affect any future name-keyed analysis of that dataset and should be fixed upstream (e.g., by appending a disambiguating instance index to the name) rather than worked around per-analysis as we did here.

VIII. CONCLUSION AND FUTURE WORK

We implemented and evaluated the two concrete extensions the original paper’s future-work section proposed in response to its own finding that multi-agent orchestration’s benefit over a single-prompt baseline is conditional, not universal. The adaptive routing controller, evaluated via leak-free cross-validation, matches or slightly exceeds both fixed strategies’ success rates at a fraction of the LLM-call cost, while also revealing that this benefit is itself benchmark-dependent — a finding consistent with, rather than resolving, the original paper’s central thesis. The patch tournament directly targets stochastic patch-generation failures via test-validated resampling, and is the stronger of the two results: aggregated across 131 newly executed local-backend repairs spanning all three benchmarks, it reaches 69.5% success versus 61.8% for the single-draw multi-agent pipeline it modifies and 64.9% for the single-prompt baseline — beating baseline outright on two of three benchmarks (QuixBugs, DebugBench) and in aggregate, at a realized LLM-call cost ($1.55\times$) close to the $1.5\times$ nominal prediction, though with a long tail of timeout-heavy bugs that inflates mean (but not median) wall-clock cost substantially. The tournament’s disagreement signal

further revealed that agreement’s relationship to success is itself benchmark-dependent rather than a fixed law, a nuance we report rather than smooth over. Future work includes combining both mechanisms (route to baseline, multi-agent, or multi-agent-with-tournament as three tiers rather than two, since the tournament’s gains here are large enough to plausibly change which bugs the router should send to the cheap path at all), extending the router’s features to non-Python languages, per-candidate timeout budgets for the tournament to bound worst-case latency, fixing the upstream DebugBench naming collision, and re-validating both extensions on Gemini once quota constraints allow.

REFERENCES

- [1] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” *arXiv preprint arXiv:2305.05176*, 2023.
- [2] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, “RouteLLM: Learning to route LLMs with preference data,” *arXiv preprint arXiv:2406.18665*, 2024.
- [3] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen, “CodeT: Code generation with generated tests,” *arXiv preprint arXiv:2207.10397*, 2022.
- [4] C. S. Xia, Y. Wei, and L. Zhang, “Automated program repair in the era of large pre-trained language models,” *Proceedings of the International Conference on Software Engineering (ICSE)*, pp. 1482–1494, 2023.
- [5] C. S. Xia and L. Zhang, “Keep the conversation going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT,” in *arXiv preprint arXiv:2304.00385*, 2023.
- [6] J. Dekoninck, M. Baader, and M. Vechev, “A unified approach to routing and cascading for LLMs,” *arXiv preprint arXiv:2410.10347*, 2024.
- [7] “Rethinking automated program repair: The impact of bug complexity, fault localization, and LLM cost-efficiency,” *arXiv preprint arXiv:2608.14065*, 2026, empirical analysis of how bug structural complexity and fault-localization precision affect APR cost-efficiency across LLMs; does not propose a routing system.
- [8] I. Bouzenia, P. Devanbu, and M. Pradel, “RepairAgent: An autonomous, LLM-based agent for program repair,” *arXiv preprint arXiv:2403.17134*, 2024.
- [9] H. Ye, M. Martinez, X. Luo, T. Zhang, and M. Monperrus, “SelfAPR: Self-supervised program repair with test execution diagnostics,” in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2022.
- [10] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [11] “CigaR: Cost-efficient program repair with LLMs,” *arXiv preprint arXiv:2402.06598*, 2024.
- [12] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Agentless: Demystifying LLM-based software engineering agents,” *arXiv preprint arXiv:2407.01489*, 2024.